ITA0612 – MACHINE LEARNING

CO4 - AT1

CODING CHALLENGE

NAME: MONIKA R

REG.NO: 192324281

Submitted To:	Submitted on:
Shri Vindhya A	13-08-2026

QUESTION: 1

Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

ANSWER:

1. K-Nearest Neighbour (KNN)

Python

```
import math
X_train = [1, 2, 3, 4, 5, 6, 7, 8]
y_train = [0, 0, 0, 0, 1, 1, 1, 1]

def sigmoid(z):
    return 1 / (1 + math.exp(-z))
def train_logistic_regression(X, y, learning_rate, epochs):
    weight = 0.0
    bias = 0.0
    n = len(X)
    for epoch in range(epochs):
        dw = 0.0
        db = 0.0
        for i in range(n):
            z = weight * X[i] + bias
            prediction = sigmoid(z)
            error = prediction - y[i]
            dw += error * X[i]
            db += error
        dw = dw / n
        db = db / n
        weight = weight - learning_rate * dw
        bias = bias - learning_rate * db
    return weight, bias
def predict(X, weight, bias):
    probabilities = []
    predictions = []
    for x in X:
        z = weight * x + bias
        probability = sigmoid(z)
        if probability >= 0.5:
            prediction = 1
        else:
            prediction = 0
        probabilities.append(probability)
        predictions.append(prediction)
    return probabilities, predictions
learning_rate = float(input("Enter learning rate: "))
epochs = int(input("Enter number of epochs: "))
weight, bias = train_logistic_regression(
    X_train,
    y_train,
    learning_rate,
    epochs
)
print("\nTraining completed!")
print("Weight =", round(weight, 4))
print("Bias =", round(bias, 4))
X_test = [2.5, 3.5, 4.5, 5.5, 6.5, 7.5]
y_test = [0, 0, 1, 1, 1, 1]
probabilities, predictions = predict(
    X_test,
    weight,
    bias
)
print("\nTest Results:")
print("---------------------------------------------")
print("Hours\tSigmoid Output\tPredicted\tActual")
print("---------------------------------------------")
for i in range(len(X_test)):
    print(
        X_test[i],
        "\t",
        round(probabilities[i], 4),
        "\t\t",
        predictions[i],
        "\t\t",
        y_test[i]
    )
correct = 0
for i in range(len(y_test)):
    if predictions[i] == y_test[i]:
        correct += 1
accuracy = (correct / len(y_test)) * 100
print("\nAccuracy =", accuracy, "%")
print("\nLearning Rate Discussion:")
if learning_rate < 0.01:
    print("Learning rate is small.")
    print("Convergence may be slow.")
elif learning_rate <= 0.1:
    print("Learning rate is moderate.")
    print("Usually provides stable convergence.")
else:
    print("Learning rate is large.")
    print("Convergence may be fast but can become unstable.")
```

Sample output

```
Enter learning rate: 3
Enter number of epochs: 5

Training completed!
Weight = 1.3866
Bias = -3.0446

Test Results:
-------------------------------------------
Hours   Sigmoid Output  Predicted       Actual
-------------------------------------------
2.5      0.6039                  1              0
3.5      0.8592                  1              0
4.5      0.9607                  1              1
5.5      0.9899                  1              1
6.5      0.9974                  1              1
7.5      0.9994                  1              1

Accuracy = 66.66666666666666 %

Learning Rate Discussion:
Learning rate is large.
Convergence may be fast but can become unstable.
```

What this program does:

- Uses a small dataset.
- Calculates **Euclidean distance** manually.
- Takes **K from the user**.
- Finds the K nearest neighbours.
- Predicts the test instance's class.
- Calculates accuracy.

QUESTION: 2

Write a Python program to implement Logistic Regression for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate
affects convergence and performance.

ANSWER:

Logistic Regression using Gradient Descent

Python

```python
import math
X_train = [1, 2, 3, 4, 5, 6, 7, 8]
y_train = [0, 0, 0, 0, 1, 1, 1, 1]
def sigmoid(z):
    return 1 / (1 + math.exp(-z))
def train_logistic_regression(X, y, learning_rate, epochs):
    weight = 0.0
    bias = 0.0
    n = len(X)
    for epoch in range(epochs):
        dw = 0.0
        db = 0.0
        for i in range(n):
            z = weight * X[i] + bias
            prediction = sigmoid(z)
            error = prediction - y[i]
            dw += error * X[i]
            db += error
        dw = dw / n
        db = db / n
        weight = weight - learning_rate * dw
        bias = bias - learning_rate * db
    return weight, bias
def predict(X, weight, bias):
    probabilities = []
    predictions = []
    for x in X:
        z = weight * x + bias
        probability = sigmoid(z)
        if probability >= 0.5:
            prediction = 1
        else:
            prediction = 0
        probabilities.append(probability)
        predictions.append(prediction)
    return probabilities, predictions
learning_rate = float(input("Enter learning rate: "))
epochs = int(input("Enter number of epochs: "))
weight, bias = train_logistic_regression(
    X_train,
    y_train,
    learning_rate,
    epochs
)
print("\nTraining completed!")
print("Weight =", round(weight, 4))
print("Bias =", round(bias, 4))
X_test = [2.5, 3.5, 4.5, 5.5, 6.5, 7.5]
y_test = [0, 0, 1, 1, 1, 1]
probabilities, predictions = predict(
    X_test,
    weight,
    bias
)
print("\nTest Results:")
print("-----------------------------------------")
print("Hours\tSigmoid Output\tPredicted\tActual")
print("-----------------------------------------")
for i in range(len(X_test)):
    print(
        X_test[i],
        "\t",
        round(probabilities[i], 4),
        "\t\t",
        predictions[i],
        "\t\t",
        y_test[i]
    )
correct = 0
for i in range(len(y_test)):
    if predictions[i] == y_test[i]:
        correct += 1
accuracy = (correct / len(y_test)) * 100
print("\nAccuracy =", accuracy, "%")
print("\nLearning Rate Discussion:")

if learning_rate < 0.01:
    print("Learning rate is small.")
    print("Convergence may be slow.")

elif learning_rate <= 0.1:
    print("Learning rate is moderate.")
    print("Usually provides stable convergence.")

else:
    print("Learning rate is large.")
    print("Convergence may be fast but can become unstable.")
```

SAMPLE INPUT AND OUTPUT:

```
Enter learning rate: 0.1
Enter number of epochs: 1000

Training completed!
Weight = 1.3224
Bias = -5.6765

Test Results:
-------------------------------------------
Hours   Sigmoid Output  Predicted       Actual
-------------------------------------------
2.5      0.0854                  0              0
3.5      0.2596                  0              0
4.5      0.5681                  1              1
5.5      0.8315                  1              1
6.5      0.9488                  1              1
7.5      0.9858                  1              1

Accuracy = 100.0 %

Learning Rate Discussion:
Learning rate is moderate.
Usually provides stable convergence.
```

This program **does not use sklearn**. It implements:

- Sigmoid function
- Gradient descent
- Learning rate
- Prediction
- Accuracy
- Sigmoid output
- Classification results

Learning Rate Discussion:

Learning rate is moderate.

Usually provides stable convergence.

Learning-rate explanation for your record/viva

Learning Rate	Effect
Very small	Training is slow and requires many epochs
Moderate	Usually gives stable convergence
Very large	May overshoot the minimum and become unstable

Sigmoid function

It converts the model's output into a value between **0 and 1**.

For classification:

Sigmoid output >= 0.5 → Class 1

Sigmoid output < 0.5 → Class 0